



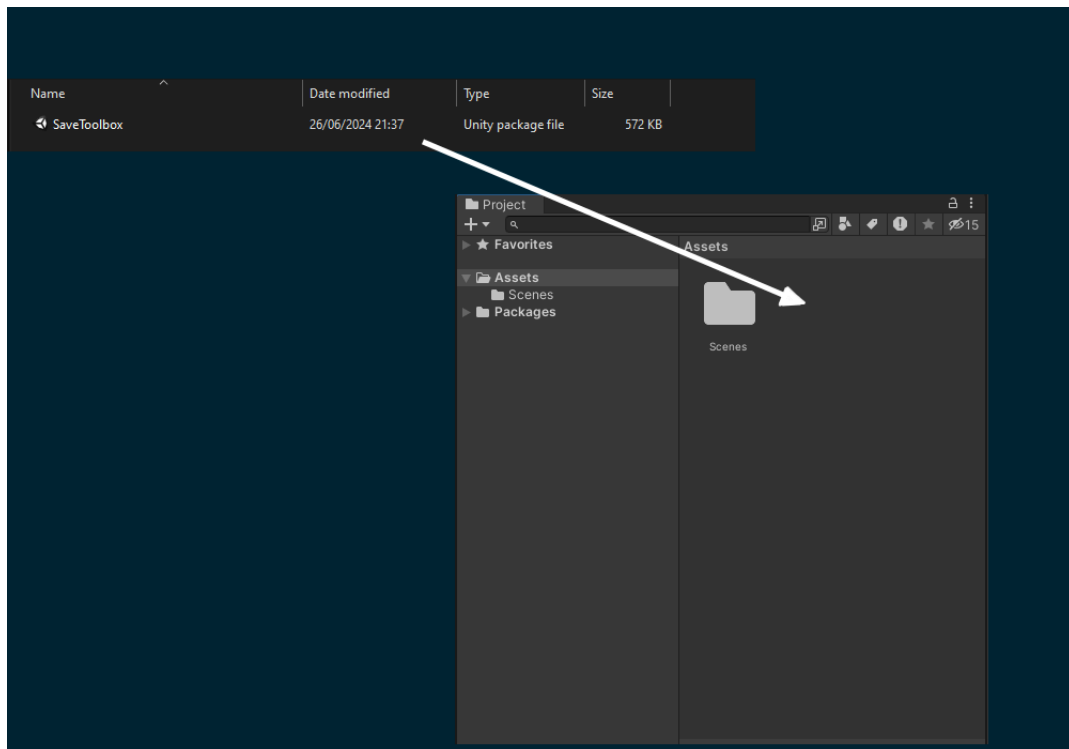
Introduction	2
Quick Start Guide	2
Saving Data	5
Save Toolbox Preferences & Settings	6
Loading Objects into the scene	8
Loadable Object Database	8
ISaveDataEntity	9
Saving Scriptable Object References	10
Scriptable Object Database	10
SaveableMonoBehaviour	11
Serialize	12
Deserialize	13
AutoSaveableMonoBehaviour	14
Saving Components	15
Stb Component Saver	16
Saving Component References	18
Save Migration	19
How Saving/Loading Pipelines Work	20
Saved Callback Handlers	21
FormerlySerializedAs	22
File Types	23
Encryption	24
Compression	25
Save Slots	26
Non-Slot Save Settings	26
Stb Data Container	27
Writing Data	27
Reading Data	27
Binary writer & Binary reader	29
Writing Data	29
Reading Data	29
Custom Save Data	30
Scene Saving	30
Extending Serializers	31
Extending Json Serializer	31
Extending Binary Serializer	34
Frequently Asked Questions	37

Introduction

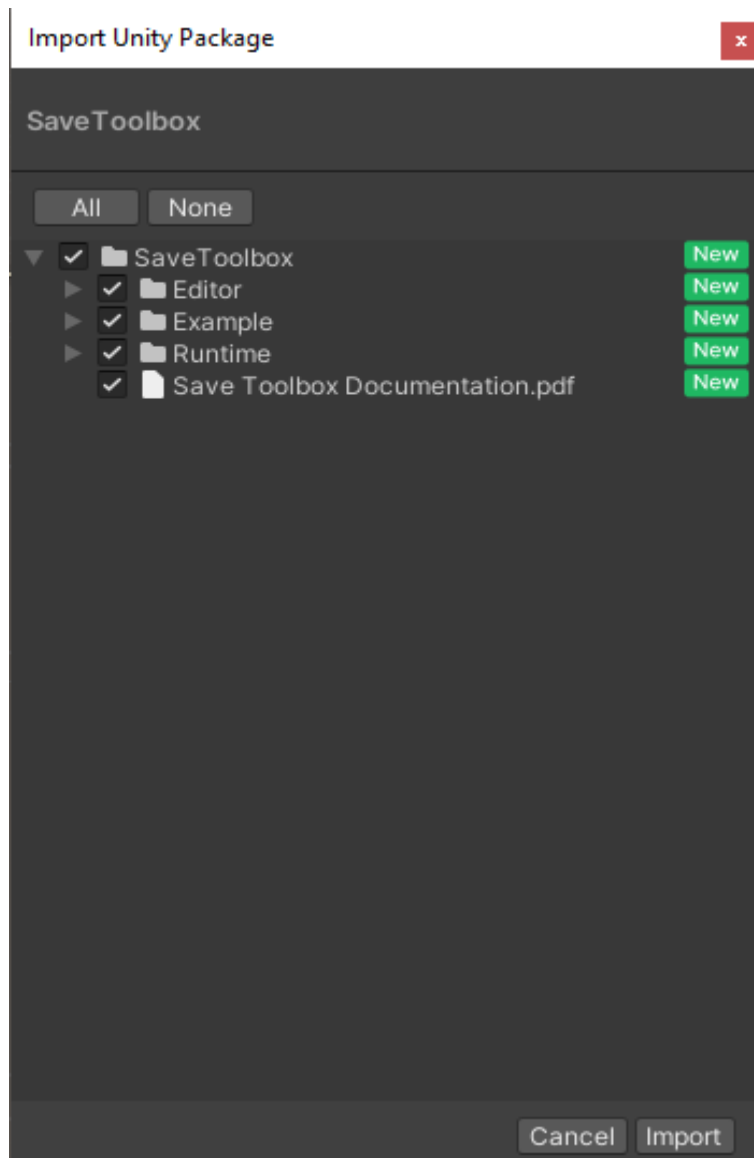
Firstly, I would like to thank you for purchasing Save Toolbox and choosing my tool for your save solution within your Unity project. Please keep an eye out on my YouTube Channel [Codevalence](#) for any tutorials I might create about the tool, they could prove very useful for your understanding! I hope the package is suitable for your needs but should there be any issues or if you have any ideas for further improvement, please do not hesitate to contact me at: codevalence@outlook.com or if you prefer feel free to join the support discord with the invite link: <https://discord.gg/jaraC5aU8Z>. Thanks again!

Quick Start Guide

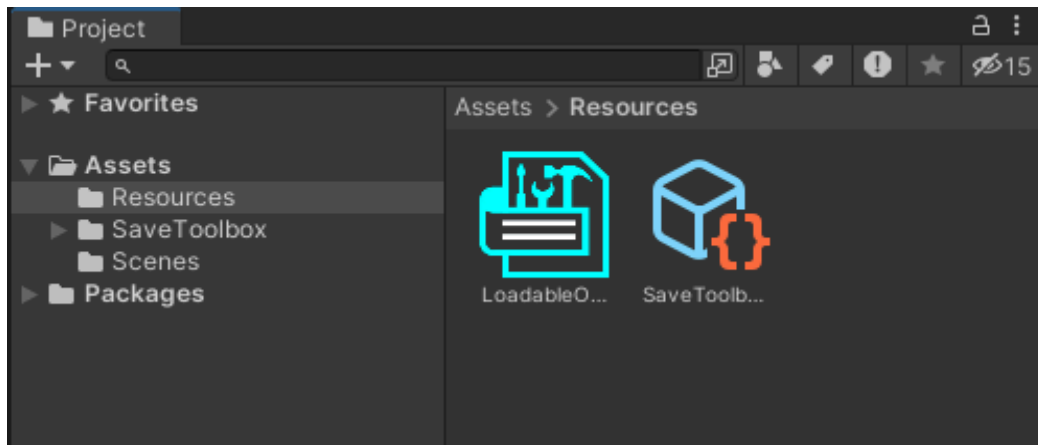
1. Download the package after purchasing it from the Unity Asset store.
2. Once downloaded, drag the SaveToolbox.unitypackage file into your unity project.



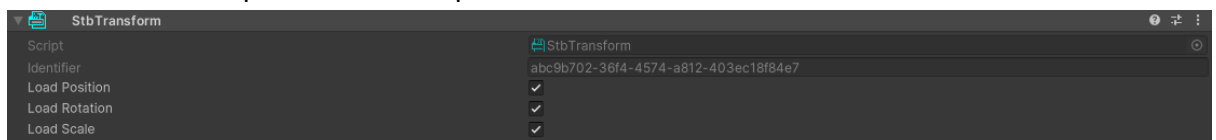
3. You should be greeted with an import window. Ensure all files are selected for import. It is recommended but not necessary to keep the example folder as it provides useful examples on how to use the tool.



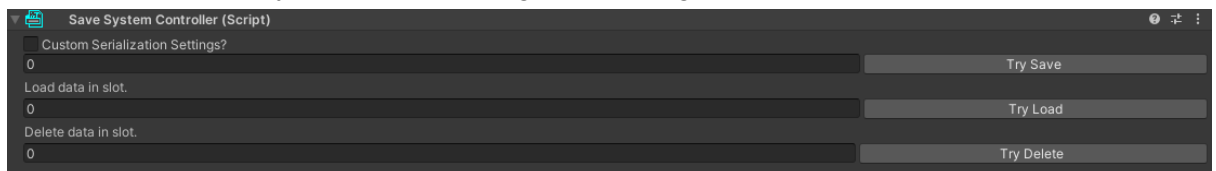
- Once imported, the save toolbox preferences file and the loadable object database should be auto generated in a resources folder. Feel free to adjust the preferences (Tools -> SaveToolbox **OR** the SaveSettings scriptable object) and check out any of the example scenes in the package. **NOTE:** You can move these files elsewhere in the project but they must be inside of a folder called "Resources". You are now free to use the tool as you see fit. The rest of the steps take you through a basic saving setup.



- Once the settings are to your liking, for a simple test you can open a scene and apply any of the MonoBehaviour save components to their relative counterparts e.g. StbTransform for saving transforms. It is recommended to use the StbCustomComponent to save data on any custom components you have made or any components that don't have a relevant save component for. Below we show the StbTransform component for example.



- For quick and easy testing in editor you can add the Save System Controller component to a gameobject in the scene. You will notice this component has save and load buttons on it, this allows you to click them in the inspector for testing the saving. It is likely you will want to call save and load from your own custom scripts, as the save system controller is just for testing in the editor. This can be done by calling SaveGame/LoadGame from the save toolbox system singleton. E.g. **SaveToolboxSystem.Instance.TrySaveGame();**. You can also use the BasicSaveUI and hook it up to unity Buttons for saving and loading.



- Press play and test out the save and load functions on the save controller to see if it saves as expected. Once you have got it working correctly feel free to implement the saving functionality this tool provides however you see fit in your project. I ad

Saving Data

Typically you will want to call the save and load functions yourself. For example, if you have a custom save/load menu in your game. There are functions on the save toolbox system singleton that are used for this:

- `SaveToolboxSystem.TrySaveGame(int saveSlotIndex);`
- `SaveToolboxSystem.TrySaveGameAsync(int saveSlotIndex);`
- `SaveToolboxSystem.TryLoadGame(int saveSlotIndex);`
- `SaveToolboxSystem.TryLoadGameAsync(int saveSlotIndex);`

The async versions of the load and save games will perform asynchronously which will spread the saving and loading over a couple of frames instead of all in 1 frame to try and avoid any lag spikes, the target frame rate can be set in the save system preferences scriptable object in your project.

There are also some simple components that can help with the auto saving of data in the game. One of these is the Save System Controller. This component has buttons on it to save data from the editor, note this will have no ability to save in a build due to it only being able to be interacted with through the inspector.

As a small helper script you can also use the BasicSaveUI.cs to call the save and load functions. You will need to reference the save and load buttons from a UI, it will call save and load when the buttons are pressed.

Please Note: To avoid a lag spike when first loading/saving your project at runtime, call the function `SaveToolboxSystem.Instance.Initialize();`. This caches all the types in your project that use `IStbSavedCallbackHandler` and `IStbLoadedCallbackHandler` etc. This can be quite intensive so it is recommended to do it in a loading screen or similar. This only needs to be done once during a runtime session. If you do not do it, it will do it automatically when you first call save or load hence the lag spike you may experience. There is a script **AutoSaveToolboxInitializer**, add this component to a gameobject in your scene and on `MonoBehaviour` awake it will automatically initialize the `SaveToolboxSystem` for you to avoid any lag spikes mid runtime.

Save Toolbox Preferences & Settings

When the SaveToolbox package is imported it will auto generate the SaveToolboxPreferences scriptable object in a resources folder within your project, find it to customize your default settings. There are various save settings that can be adjusted. Please understand that these settings will only be implemented on saves to the game and not any save data you create custom through the SaveToolbox pipeline (they can be set in the FileSaveSettings parameter instead). Here is a list of all the settings and what they do:

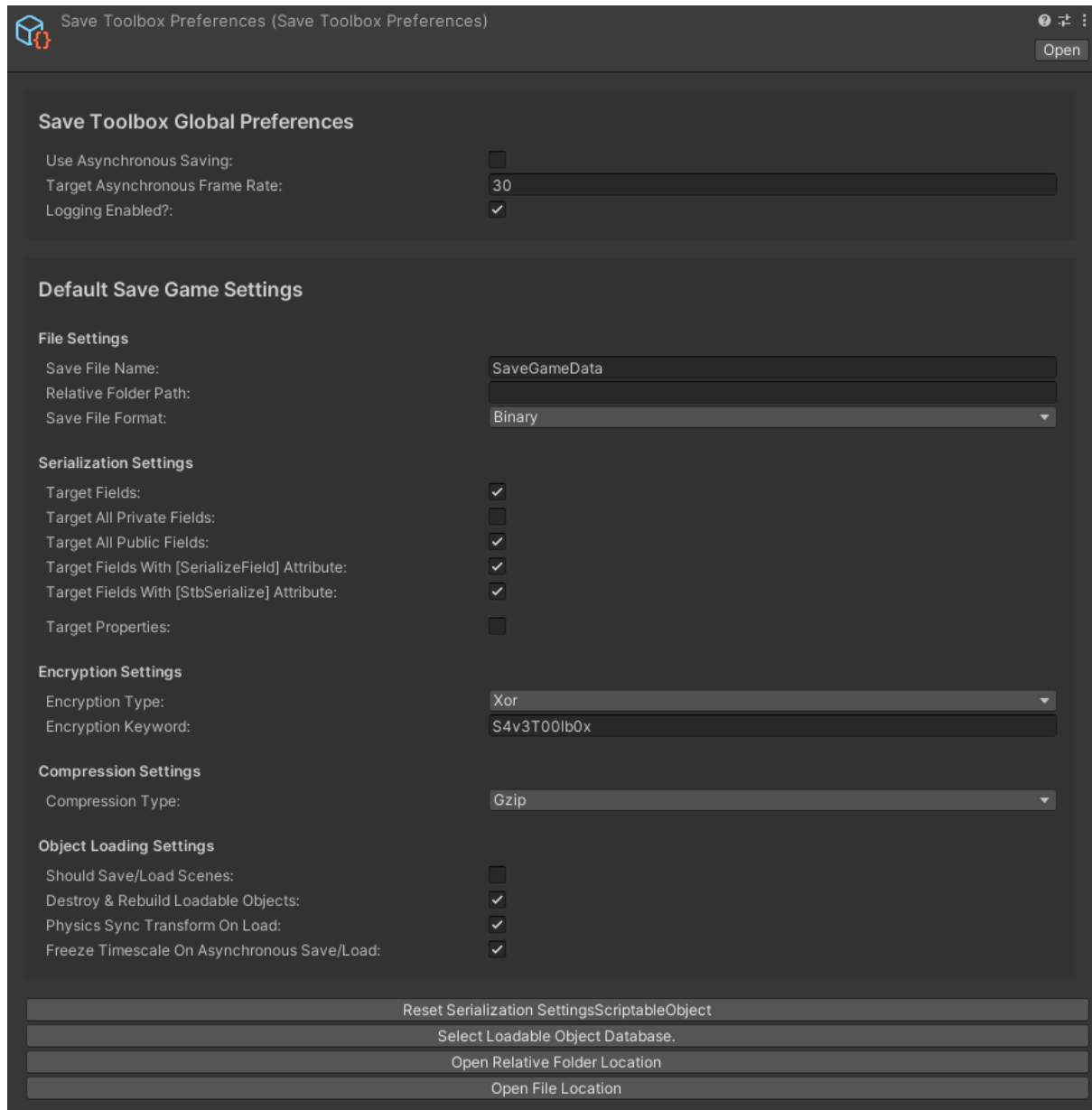
- **Save File Name** - The name that the file will save with.
- **Relative Folder Path** - The folder path of the save file, this originates from the Application.PersistentDataPath.
- **Save File Format** - The file type the save data will be saved as.
- **JSON Pretty Print** - If the file type chosen is json do you want pretty print enabled? This includes white space and newlines in the file to make it more readable, it is recommended.
- **Serialization Settings** - Allows the user to decide how objects that are passed in to be saved are handled. You can choose if it saves data of fields or properties or both. It is recommended to not change these settings unless you are confident in the settings you are choosing.
- **Encryption Type** - The type of encryption to be used. Xor is a simpler form of encryption that uses a key. Aes is recommended, it also uses a key as well as an initialization vector for additional security (save toolbox can generate this for you).
- **Encryption Keyword** - The keyword that will be used for the encrypting of the save data. It is used to decrypt and encrypt the data, making sure to keep it secret.
- **Initialization Vector** - Another layer of security for Aes encryption. Save Toolbox generates this for you, you can regenerate it by pressing the regenerate button below the field.
- **Compression Type** - The type of compression the file will have. Compression allows for smaller file sizes.
- **Should Save Scene** - Do you want SaveToolbox to handle the saving and loading of current scenes? If selected when a file is loaded it will find all necessary scenes and load them.
- **Destroy & Rebuild Loadable Objects** - When loadable objects are spawned in the scene, if an instance already exists, should it destroy the existing instance and create a new one. Recommended to keep it enabled even though it is slower to prevent any unwanted data existing between save states.
- **Physics sync transform on load** - If you are creating a project with physics objects, set this to enabled and it will ensure the position is correct on load for the physics objects, unnecessary for projects without physics objects.
- **Scene Lookup** - If we are saving the scenes, we need a way to store reference to the scene. Build index means we find the scene by build index, changing this in build settings after a save has been created can lead to losing reference to the scene. Path means we find the scene by the asset path, moving the scene asset can lead to the toolbox not being able to find the scene.
- **Asynchronous Saving** - Allows saving and loading to be done over multiple frames to avoid large lag spikes. When changing this value, it will cause a recompile, a short

amount of time after as it applies a scripting define in your project so please be aware of this.

- **Freeze Timescale On Asynchronous Save/Load** - Freezes timescale when saving and loading asynchronous to ensure object states don't change before loading/saving has finished. Recommended to keep this as enabled.
- **Target Asynchronous Frame Rate** - If asynchronous saving is enabled, what is the lowest acceptable frame rate.
- **Logging Enabled** - Save Toolbox has built in logs for when save or load is executed etc, if this is enabled it will display those logs in the console.

As the root of the save path is the Application.PersistentDataPath it can change from devices of different platforms, so please be aware of this.

Below find an example image of the Save Toolbox Preferences



Loading Objects into the scene

In order to load an object into the scene which was spawned whilst in play mode before it was saved. You will need to ensure the object was a prefab with a LoadableObject component on the game object. You will also need to ensure this prefab is present in the LoadableObjectDatabase. This can be done by creating a prefab with the LoadableObject component in your assets and pressing refresh on the LoadableObjectDatabase scriptable object or in the case refresh doesn't work you can press rebuild (This should only be used in situations where it is broken, see LoadableObjectDatabase section for a detailed explanation).

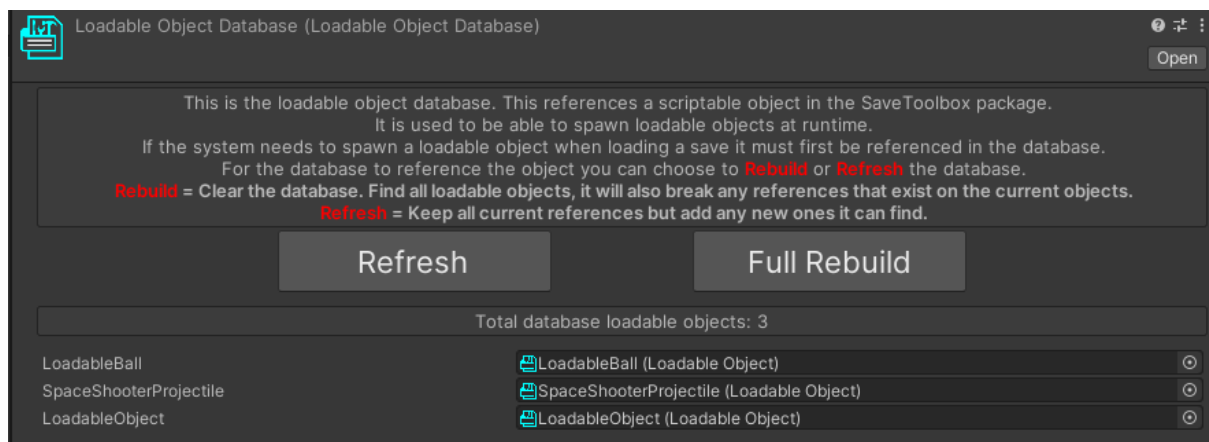
Ensure that any custom data on your loadable object that you want to be loaded has its own saving component made by you **or** uses StbComponentSaver **or** a specific StbMonobehaviourComponent.

Loadable Object Database

The loadable object database is a scriptable object singleton that holds all references to any prefab that can be loaded into a scene at runtime. Any object that is put into the loadable object database must have the LoadableObject.cs component attached to it.

To update the database so it has all current loadable objects in the database you can find the database in the project "LoadableObjectDatabase.asset" or you can go to Window -> Save Toolbox open the loadable object database tab.

Once you have either window open, you can select the refresh database, this will look for any loadable objects that are not currently in the database. When a loadable object is put into the database an id on the loadable object is set to its index in the database. If you are having errors finding any new loadable objects you can press rebuild database. **Please note:** Rebuilding a database may invalidate any older saves that worked off of the old database values. If you have leftover old or missing prefabs in the database that you no longer want there, it may not be worth rebuilding to remove them if you have already released your project as it could invalidate old saves, it should still work with null references nevertheless.



ISaveDataEntity

ISaveDataEntity is an interface that has the API of saveable objects in the Save Toolbox. They are the core of how the saving works, all of the MonoBehaviours and class instances that are collected when the game is saved are ISaveDataEntities.

It has a string "SaveIdentifier", which is used to determine to which object the data goes when it is loaded, as such they should always be unique. If you are creating a MonoBehaviour implementing this interface it will be collected by the save system automatically however if you create an instance of a **non-MonoBehaviour** class the system has no way to find it so you should add it to the static list in the SaveToolboxSystem using **SaveToolboxSystem.AddSaveGameEntity(ISaveDataEntity saveDataEntity)**.

It is highly recommended if you want to save MonoBehaviours to not implement this interface but inherit from [SaveEntityBehaviour](#) which is described in the next section as it handles unique save identifier generation for the user automatically. In **non-AutoSaveableMonoBehaviour** instances the save identifier will have to be set manually.

It also caches the LoadableObjectId which stores which loadable object it belongs to if it was spawned into the scene at runtime. There are also the Serialize and Deserialize functions on this interface that will need to be implemented that dictate how the data will be handled, which is explained in the [Serialize section](#).

```
[ExecuteAlways]
public interface ISaveDataEntity
{
    string Identifier { get; set; }

    public string LoadableObjectId { get; set; }
    public bool IsPartOfLoadableObject => !string.IsNullOrEmpty(LoadableObjectId);

    object Serialize();
    void Deserialize(object data);

    public void GenerateNewIdentifier()
    {
        Identifier = Guid.NewGuid().ToString();
    }

    public void AddEntityToSaveData()
    {
        SaveSystem.AddSaveGameDataEntity(this);
    }

    public void RemoveEntityFromSaveData()
    {
        SaveSystem.RemoveSaveGameDataEntity(this);
    }
}
```

Saving Scriptable Object References

This package supports saving references to scriptable objects. This includes lists and arrays of scriptable objects. However in order to do so your scriptable object must be referenced in the scriptable object database.

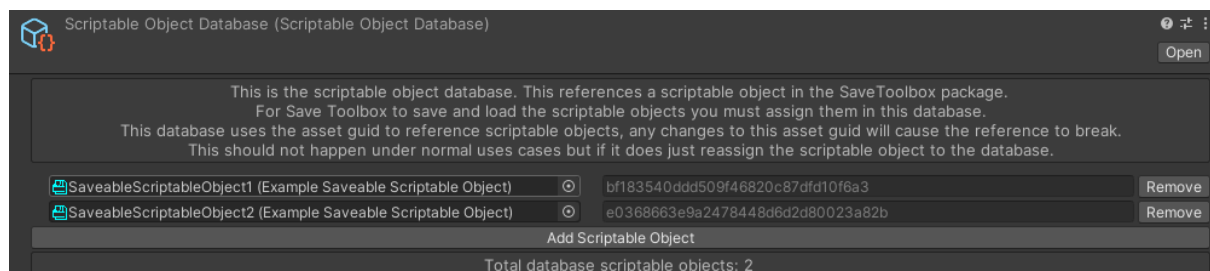
Scriptable Object Database

Similar to the Loadable Object Database in order to save references to any scriptable objects you may have, the scriptable object database must have references to them.

In order to populate the database just click the add scriptable object button and drag a scriptable object into the object field. When a scriptable object field has been dragged in, the asset guid for that scriptable object will populate the string field next to it, this should be read only. The asset guid field is for debug purposes if a save should break so you may find the scriptable object reference that has broken. If you try to save a reference to a ScriptableObject that is not in the database it will load a null reference when it loads. **Please Note:** Editing the Scriptable Object Database at runtime is NOT recommended and may cause issues.

This database will auto generate in a resources folder when you first import the package, feel free to move it but it **MUST** be inside a resources folder to work correctly, similar to the [LoadableObjectDatabase](#).

Please Note: Manually going into a scriptable object file and changing it's asset guid will break the way the database loads the scriptable object and any existing saves that rely on this. If you find yourself in the situation, deleting the old entry in the database and adding the scriptable object again should solve the issue.



SaveableMonoBehaviour

The saveable MonoBehaviour is a helper class that can be inherited from instead of MonoBehaviour that implements the ISaveDataEntity API.

It generates a unique save identifier for you when it is instantiated as well as caching if it is part of a loadable object automatically. It will also set itself within the Loadable object, it is **highly** recommended to use this if creating custom MonoBehaviours that save data but in cases where it cannot be done the ISaveDataEntity can be implemented instead.

Inheriting from SaveableMonoBehaviour will require you to override the Serialize and Deserialize functions which will determine how you handle the saving and loading of data on the component. If you have a custom component that you want to add custom saving to that inherits from MonoBehaviour just replace MonoBehaviour with SaveableMonoBehaviour and it should work as intended once the Serialize and Deserialize functions have been implemented.

```
[ExecuteAlways]
public abstract class SaveableMonoBehaviour : MonoBehaviour, ISaveDataEntity, ISaveEntityLifecycle
{
    [SerializeField, ReadOnly]
    public string Identifier { get; set; } = Guid.NewGuid().ToString();

    public string LoadableObjectId { get; set; }

    public abstract object Serialize();
    public abstract void Deserialize(object data);

    /// <summary>
    /// This is to ensure that when we add the component to the gameobject it registers itself with the loadable object.
    /// This should also happen at runtime due to the class attribute "ExecuteAlways".
    /// </summary>
    private void Reset()
    {
        Identifier = Guid.NewGuid().ToString();

        var loadableObjects :LoadableObject[] = transform.root.GetComponentsInChildren<LoadableObject>();
        foreach (var loadableObject in loadableObjects)
        {
            loadableObject.UpdateReferencedBehaviours();
        }
    }

    [ContextMenu("Refresh Identifier (WARNING: Can break existing saves.)")]
    private void RefreshIdentifier()
    {
        Identifier = Guid.NewGuid().ToString();
    }

    public virtual void OnSaveCompleted() {}
    public virtual void OnLoadingSpawned() {}
    public virtual void OnLoadCompleted() {}
}
```

Serialize

The serialize function **public override object Serialize()**. This function expects an object that will be serializable to be returned. All primitives are serializable by default, if a type is returned that doesn't have custom serialization functionality set up in the serializers or an [extension of the serializer](#) it will then go through each serializable field and collect the data off of it. **Please Note:** The fields/properties that have their data collected are determined by the serialization settings used, unless explicitly specified it will use the default serialization settings stored in your SaveToolboxPreferences asset that is automatically generated in a resources folder in your project. Here you can choose if you target fields/properties fields with specific attributes etc. You can read more about it in the [Settings](#) section.

Custom data classes are a good way to store data for different objects, however some classes you will be able to just inherit from SaveableBehaviour or implement the ISaveDataEntity interface and as long as it is serializable and has public getters/setters for data it can be gathered and reapplied. **Please Note:** on deserializing an instance of the object returned in Serialize is created so if it is an object that has functionality on creation this should be taken into consideration.

We have a custom class **TransformSaveData** that is Serializable. It holds data such as position rotation and scale as well as a boolean that determines if it should save this data. When serializing this class instance is just returned with all the data set on it. **Please Note:** Any custom serialization data objects must have a default constructor (A constructor with no parameters) if it is **not** a struct. Structs will work regardless. Below is an example of an implemented Serialize() function.

```
public class StbTransform : SaveableMonoBehaviour
{
    [SerializeField]
    private bool loadPosition = true;

    [SerializeField]
    private bool loadRotation = true;

    [SerializeField]
    private bool loadScale = true;

    public override object Serialize()
    {
        var transformData = transform;
        var saveData = new TransformSaveData( savePosition: loadPosition, position: transformData.localPosition,
        saveRotation: loadRotation, transformData.localRotation, saveScale: loadScale,
        scale: transformData.localScale);
        return saveData;
    }
}
```

Deserialize

When deserializing you will be passed back an object of type that you passed it during serialization so in the case of the StbTransform component you will receive an object of type **TransformSaveData**.

This will be of type object so it will need to be cast to the correct type before it is used like is done in the example below. From there, the data is gathered from the object and applied to their respective places and in this case to the transform of the gameobject.

```
public override void Deserialize(object data)
{
    if (data is TransformSaveData saveData)
    {
        loadPosition = saveData.SavePosition;
        loadRotation = saveData.SaveRotation;
        loadScale = saveData.SaveScale;

        // Apply values if need be.
        if (loadPosition) transform.localPosition = saveData.Position;
        if (loadRotation) transform.localRotation = saveData.Rotation;
        if (loadScale) transform.localScale = saveData.Scale;
    }
}
```

AutoSaveableMonoBehaviour

Similar to the [SaveableMonoBehaviour](#) the AutoSaveableMonoBehaviour is an abstract class that can be inherited from. Unlike the [SaveableBehaviour](#) however it does not require inheriting classes to implement their own Serialize and Deserialize functions. Instead this abstract class handles it for you. It will serialize all data that is targeted by the Serialization settings on the class that inherits from AutoSaveableMonoBehaviour automatically.

It also automatically reapplies all this data on to the AutoSaveableMonoBehaviour on Deserialize. This ensures a super easy implementation for MonoBehaviours that have data on them that the user would like to easily save. However, as the serialize and deserialize process is automatic, custom functionality can not be implemented into the data when it serialized/deserialized. Also, if you want to save data that is not directly stored on this MonoBehaviour you should implement [SaveableMonoBehaviour](#) instead as this is more extendable and allows for more customizability.

Saving Components

There are a variety of components for easy saving of specific components in unity. These components will hold a reference to their targeted type and save the data on them. These components can be replicated with the [StbComponentSaver](#), which is the best way for saving custom scripts without the use of code. However these have been set up for common components as they can be easier to understand for most people.

All the components are prefixed with “Stb” for ease of finding. If there isn’t a saving component equivalent for a component you want to save you should use the [StbComponentSaver](#) or create your own [SaveableBehaviour](#) or implement [ISaveDataEntity](#).

- **StbCamera** - Saves data for the camera such as depth and other camera settings.
- **StbCanvasGroup** - Saves data for the canvas group such as blocking raycasts and alpha.
- **StbDropdown** - Saves data for a UI dropdown component such as its currently selected element.
- **StbGameObject** - Saves whether an object is currently active or not as well as tag and other data.
- **StbNavMeshAgent** - Saves data from a nav mesh agent such as velocity, speed, target etc.
- **StbLight** - Saves data about a light such as type, range, intensity etc.
- **StbRigidBody** - Saves RigidBody data such as its current velocity.
- **StbRigidBody2D** - Saves RigidBody2D data such as its current velocity.
- **StbSlider** - Saves the current value of a UI Slider.
- **StbTextField** - Saves the current text of a text field as well as some settings.
- **StbToggle** - Saves the current value of a UI Toggle.
- **StbTransform** - Saves the current transform values of an object.

Stb Component Saver

The Component Saver is a very useful component within the library. It allows you to save any data on any custom component you may have created or exists in unity by default as long as it has serializable data. It allows you to save data without the need to write any code.

Please Note: Programming your own way to store data etc can be more efficient but for the saving of data without needing to code this is a very powerful tool and is highly recommended.

The way it works is through reflection. It will analyze the code of the components it has serialized in the inspector and search through the component until it finds all the fields and properties it can save. You can then select through all these fields and properties and choose the ones which you want their data to be saved. When the game is saved it will then save all the current states of these fields/properties. When loaded it will set the data to that which was saved for each specific field/property.

Below, you can see an example of how it is set up to save the data of a Rigidbody and a BoxCollider. **Please Note:** The component is set up to target more than one component, so there's no harm in doing so. Although it can target components from different gameobjects it is highly recommended to keep it only targeting components on the same object especially if they are spawned in as it could lose reference to its target if it was spawned in on a different object.

Here is an example of how to set it up:

1. Add the Stb Component Saver component on to a gameobject.
2. Add the target component that has data on which you would like to save to the same game object.
2. Then click the “add component” button on the Stb Component Saver component.
3. Drag the target component into the component reference field.
4. Click the add field button. A drop down box should appear.
5. From the dropdown box you can select the field you would like to save.

Now, when the game saves it should save the fields you selected and load the saved data into the fields when you load the game. Below is a visual example of a setup.

StbComponentSaver

Identifier:
Component:
Rigidbody
Component Identifier
Deserialization Priority
Fields To Save
Rigidbody field 1:
Use A Custom Component Saver?

c734ffe1-14c4-4a37-b416-011e1c42fe67
GameObject (1) (Rigidbody)
c3f15260-6b73-4256-9b00-164c3e326590
0
velocity
RigidbodyComponentSaver

Remove Component
Remove Field
Add Field
Add Custom Saver

Component:
BoxCollider
Component Identifier
Deserialization Priority
Fields To Save
BoxCollider field 1:
BoxCollider field 2:

GameObject (1) (Box Collider)
9cd3fac0-07cb-4ad2-a8a3-1a1e14109b9d
0
isTrigger
enabled

Remove Component
Remove Field
Remove Field
Add Field

Add A New Component

Rigidbody

Mass
Drag
Angular Drag
Automatic Center Of Mass
Automatic Tensor
Use Gravity
Is Kinematic
Interpolate
Collision Detection
Constraints
Layer Overrides
Include Layers
Exclude Layers

1
0
0.05
✓
✓
✓
☐
None
Discrete
Nothing
Nothing

Box Collider

Edit Collider
Is Trigger
Provides Contacts
Material
Center
Size
Layer Overrides

☐
☐
None (Physic Material)
X 0
Y 0
Z 0
X 1
Y 1
Z 1

Add Component

Saving Component References

In SaveToolbox you are able to save component references to other components that implement the `ISaveDataEntity` interface. The reason it needs to implement this interface is so that the saving/loading system can use the “Save Identifier” on it to locate the component when all data is loaded. All built in MonoBehaviour save components in SaveToolbox like “StbTransform” or “StbGameObject” should implement it so should work straight out of the box. Also any class that the user creates that inherits from `SaveableMonoBehaviour` or `AutoSaveableMonoBehaviour` will implement `ISaveDataEntity` automatically and should be able to have its reference saved.

In order to save a reference to a component that implements the `ISaveDataEntity` interface you must reference it using the `SaveEntityReference<T>` class. Where T is the type of `ISaveDataEntity` component you want to reference. See below an example for saving a reference to an `StbTransform`. This class will allow you to set the reference in the inspector. You can also store reference to arrays of components implementing `ISaveDataEntity`.

```
[SerializeField]
private SaveEntityReference<StbTransform> saveTransform;

[SerializeField]
private SaveEntityReference<StbGameObject>[] stbGameObjectsArray;
```

When you want to get the reference cast to its respective type just use `.Reference`. In the case below it will return an object of type `StbTransform`.

```
var loadedStbTransform = saveTransform.Reference;
```

Please Note: If you want to use data on the referenced component after loading, it is better to use the [IStbLoadedCallbackHandler](#) rather than a `Deserialize()` function to ensure all the data on the reference component has been loaded. As the `IStbLoadedCallbackHandler.HandleDataLoaded` is called after all components have received their respective data.

Save Migration

Save migration in Save Toolbox is a very basic implementation when we load a save we try to migrate it if necessary. There is an example save migrator, in the example folder of the package. Inside the script, we check which version of the save data it is, this is stored by default on the save slot metadata. As an example we will increment this version number everytime we change how the save data is organized.

If the version is different in the example we take all the TransformSaveDatas and reset their values if the version number isn't what is expected. Of course, this implementation has no real use but it is an example of how you can implement your own changes. E.g. You have a save with player data and in that data you have a field called walk speed, now you changed the player to only be able to fly now but you want to migrate saves so their old walk speed is proportional to their new fly speed. This is the system you would use.

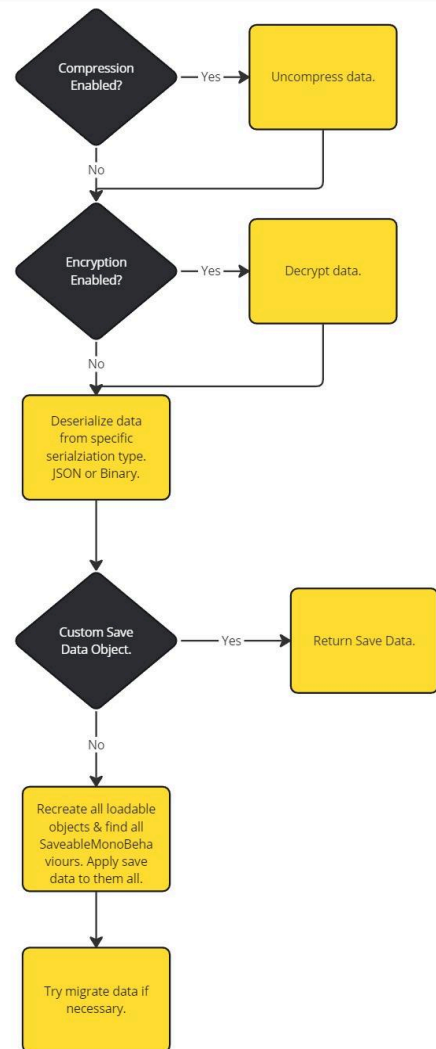
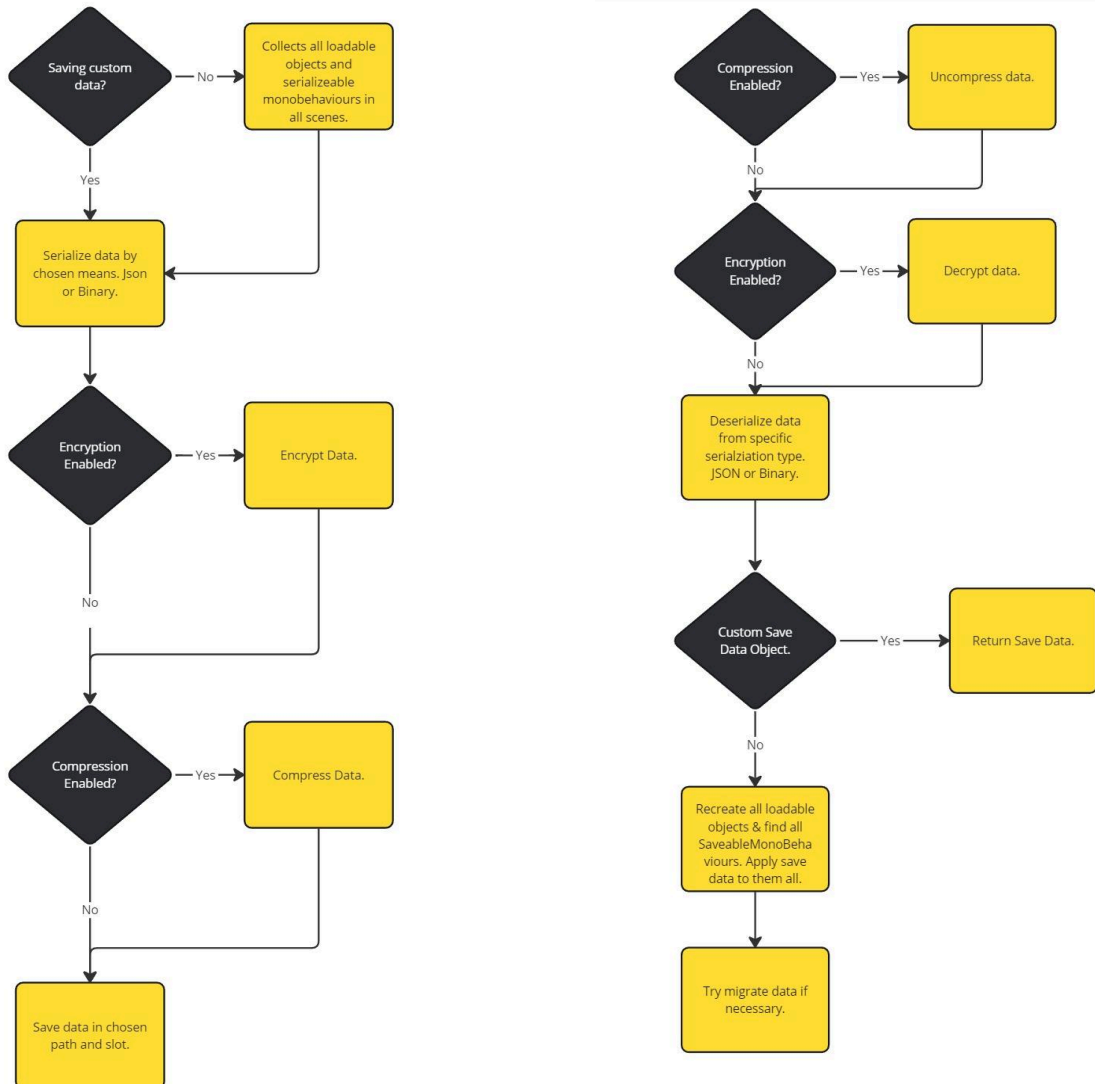
In the example I am using the built in metadata save version but realistically you can set up your own versioning system if desired. To set the current save migrator make sure to set the public static "saveMigrator" field on the SaveToolboxSystem.cs singleton to your own desired migrator that inherits from AbstractSaveMigrator.

```
public class ExampleSaveMigrator : AbstractSaveMigrator
{
    public override SlotSaveData Migrate(SlotSaveData slotSaveData)
    {
        if (slotSaveData.SaveMetaData.SaveVersion == 0.0)
        {
            foreach (var loadableObjectSaveData in slotSaveData.SaveData.saveDataEntityObjects)
            {
                if (loadableObjectSaveData.ObjectValue is TransformSaveData transformSaveData)
                {
                    // Reset all transform save data.
                    loadableObjectSaveData.ObjectValue = new TransformSaveData();
                }
            }
        }

        return slotSaveData;
    }
}
```

How Saving/Loading Pipelines Work

This section will detail how the saving pipeline works from a basic point of view. Below is a flowchart representation of what happens.



Saved Callback Handlers

Sometimes you save the game data and you want a callback for this in your code that isn't itself actually saved. E.g. You want a popup to appear when you have saved your game data.

If you want your class to receive the callback, make sure it implements the interface `IStbSavedCallbackHandler`, it will force you to implement the function `HandleDataSaved` which passes the save data as a parameter. Whenever you save the game data this function will be called.

This will work automatically with any `MonoBehaviours` that implement the interface but for non-`MonoBehaviours` they need to self subscribe to a static list and unsubscribe themselves from the list when they should no longer receive the callback. There is a `NonMonoBehaviourCallbackExample.cs` script. There is also the `IStbLoadedCallbackHandler` that handles the callback when save game data is loaded. Below you can see an example of the `IStbSaveCallbackHandler` and `IStbLoadedCallbackHandler`.

```
public class NonMonoBehaviourCallbackExample : IStbSavedCallbackHandler, IStbLoadedCallbackHandler
{
    public void HandleAdded()
    {
        SaveToolboxSystem.Instance.StbCallbackDistributor.AddNonMonoBehaviourSaveCallbackHandler(this);
        SaveToolboxSystem.Instance.StbCallbackDistributor.AddNonMonoBehaviourLoadCallbackHandler(this);
    }

    void IStbSavedCallbackHandler.HandleDataSaved(SaveData saveData)
    {
        if (SaveToolboxPreferences.Instance.LoggingEnabled) Debug.Log(message: "Non-MonoBehaviour Saved callback received.");
    }

    void IStbLoadedCallbackHandler.HandleDataLoaded(SaveData saveData)
    {
        if (SaveToolboxPreferences.Instance.LoggingEnabled) Debug.Log(message: "Non-MonoBehaviour Loaded callback received.");
    }

    public void HandleRemoved()
    {
        SaveToolboxSystem.Instance.StbCallbackDistributor.RemoveNonMonoBehaviourSaveCallbackHandler(this);
        SaveToolboxSystem.Instance.StbCallbackDistributor.RemoveNonMonoBehaviourLoadCallbackHandler(this);
    }
}
```

FormerlySerializedAs

If you have any pre-existing saves or users have pre-existing saves, if a class/struct that is saved has its name or assembly changed then you must use the `StbFormerlySerializeAs` attribute on the class/struct in order to ensure that older saves work.

This would only really be necessary if you didn't want older saves to break. There is also the possibility of using [SaveMigration](#) to rectify this but this should also be a valid solution. In order for this to work you need to attach the attribute to the class/struct with the old assembly path as a parameter. This will then be able to be checked against the old save data to find out what data structure it was prior.

If you are unsure what the old assembly path is, you can find it in the old save data if it is JSON and readable or you can use the function `AssemblyQualifiedName` on a type before you change it's assembly/namespace e.g. `"typeof(OldData Type).AssemblyQualifiedName"`. Below is an example of how it can be implemented.

```
[Serializable, StbFormerlySerializedAs("SaveToolbox.Runtime.Example.PreviousDataExample, SaveToolbox, Version=0.0.0.0, Culture=neutral, PublicKeyToken=null")]
public struct PreviouslySerializedAsExampleData
{
    public float health;
    public float damage;

    public PreviouslySerializedAsExampleData(float health, float damage)
    {
        this.health = health;
        this.damage = damage;
    }
}
```

File Types

There are currently 2 file types available to save to: JSON or binary. Below find a table describing the advantages and disadvantages of the file types. The file type can be set in the SaveSettings file or in Tools -> SaveToolbox. It should also be noted that both these file types are created from custom serializers that have been made specifically for this package.

JSON	BINARY
Human readable, good if you want to allow yourself or others to read through a save file to find errors or to edit their save by changing values.	Not human readable. Good if you don't want people to easily edit the file. Although encryption is also recommended if you want to prevent edits.
Typically larger files.	Typically smaller files.
Serializes much slower than its binary counterpart.	Binary tends to be quicker when serializing but this will be noticeable with large save files.

Encryption

There is encryption built into the tool that allows you to encrypt any data saved and decrypt any data loaded from disk. There are two different forms of encryption that can be used in this tool: Xor & Aer.

Please note: If you have chosen JSON as your file type to save for its readability it will lose its readability if it is encrypted. I would recommend reading up on both encryption methods but otherwise I have created a simple table to outline the differences below.

Xor	Aes
Uses an encryption keyword.	Uses an encryption keyword and an initialization vector.
Simpler and easy to understand. Uses bitwise operations.	Complex and harder to understand.
Efficient as it is only doing bitwise operations.	More taxing on performance although it shouldn't make much of a difference in most usual cases.
Weaker security, if someone has the keyword it can be decrypted.	Stronger security.

Compression

Compression is a good way to reduce your file sizes. I would recommend only doing this if your file size is starting to become a concern. Currently there is only one form of compression, GZip. GZip is a common form of compression. Feel free to use this to make your save files smaller, may not be necessary with already small save files.

Feel free to use encryption **AND** compression on your save data if necessary. **Please note:** If you have chosen JSON as your file type to save for its readability it will lose its readability if it is compressed and will likely not be editable from the file.

Save Slots

The concept of a save slot is to define different instances of the same game saves in different save slots. The SaveToolbox save slots are directory based which means each save slot has its own folder. Whenever you save a game instance it will request a slot index, by default it is 0, you can have as many slots as you would like but if it is set to -1 it will not save the data in any slot and instead exist outside of the slot directories, this is explained in more detail below.

Non-Slot Save Settings

Sometimes you may save data without a save slot. This is useful for data you will want to persist between different save instances of a game such as player settings. You will have to do this explicitly when you are saving by not providing a slot index or a slot index that is -1 in the file save settings when you create save data.

Stb Data Container

The StbDataContainer is another way to serialize your save data. It allows you to put any object data into a container as a key value, similar to a dictionary.

Writing Data

Writing data to the StbDataContainer is simple, you call the function:

```
dataContainer.AddKeyValueEntry("key", objectvalue);
```

In the example below we return the data container on serialize after we have added some key values. The data is not just limited to primitive types you can also pass objects in here as well as long as they are serializable.

```
public override object Serialize()
{
    var dataContainer = new StbDataContainer();
    dataContainer.AddKeyValueEntry("exampleBool", exampleBool);
    dataContainer.AddKeyValueEntry("exampleString", exampleString);
    dataContainer.AddKeyValueEntry("exampleChar", exampleChar);
    dataContainer.AddKeyValueEntry("customData", customData);

    return dataContainer;
}
```

Reading Data

The below example is a continuation of the Serialize function above. This StbDataContainer has been received through the Deserialize function on an object implementing ISaveDataEntity such as SaveableBehaviour as a load game was called. To get the data off of the data container we must first cast the object "data" to a data container like is done in the image below. Once it is cast, you can call the function:

```
dataContainer.GetValue<T>("key");
```

T in this context is the type of object you are looking for such as an int or a custom class etc. The key is the key that was assigned to it when serializing. The return value of this function will be the object's value that matches the key.

```
public override void Deserialize(object data)
{
    var dataContainer = (StbDataContainer)data;
    exampleBool = dataContainer.GetValue<bool>(key: "exampleBool");
    exampleString = dataContainer.GetValue<string>(key: "exampleString");
    exampleChar = dataContainer.GetValue<char>(key: "exampleChar");
    customData = dataContainer.GetValue<CustomData>(key: "CustomData");
}
```

Binary writer & Binary reader

Writing Data

Writing data to the `StbByteWriter` is simple and lightweight. You must remember that whatever you write must be read back in the same order and anything that is written must be read back to get to the next value. When serializing using an `StbByteWriter` you can just call `Write()`, for all primitives and some common unity types like vectors and quaternions. Collections are also supported for each type (Queue, Stack, List, Array & Dictionaries), for writing more complex types you must write it field by field in terms of the primitive data you want to be saved.

Please Note: When serializing data using the `StbByteWriter` you can return the byte array using `GetByteArray()` and this will cause a `byte[]` to be returned on `Deserialize()`, like in the examples below. Or you can just return the byte writer itself and a byte reader with the array in will be returned on `deserialize`, whichever is preferred by the user.

```
public override object Serialize()
{
    var byteWriter = new StbByteWriter();
    byteWriter.Write(exampleBool);
    byteWriter.Write(exampleString);
    byteWriter.Write(exampleInt);
    return byteWriter.GetByteArray();
}
```

Reading Data

Reading the data is very similar to writing except now you must use `.Read` functions like below and you need to be explicit about the types such as `ReadInt`, `ReadBool`, `ReadStringArray` etc. It must be read in the same order it was written or it will break the deserialization process. Below a `byte[]` is returned in this example as the `byteWriter` array was serialized and not the `byteWriter` itself which would have otherwise given us an `StbByteReader`. Instead we cast the data to a `byte[]` and use it to construct a new byte reader, which can then be read from. The return values of the read functions are then assigned to their correct fields.

```
public override void Deserialize(object data)
{
    var byteData = (byte[])data;
    var byteReader = new StbByteReader(byteData);
    exampleBool = byteReader.ReadBool();
    exampleString = byteReader.ReadString();
    exampleInt = byteReader.ReadInt();
}
```

Custom Save Data

Along with the ability to save the game state you can also pass in any serializable object to save manually. This can be done by calling the TryCreateSaveData function on the SaveToolboxSystem.cs singleton. Let's say you are saving your game data as json but you have a bunch of world position vectors that you would not like to save in json, but instead binary. You can pass this list of positions to the save system as follows.

```
SaveToolboxSystem.Instance.TryCreateSaveData(saveObject, fileSaveSettings);
```

The saveDataArgs will need to be created with all your desired settings for saving this object. **Please Note:** As this isn't a game save data and instead custom it will not use the default save settings set in the asset SaveSettings instead you have to define these yourself and pass them in through the saveDataArgs parameter. The loading will also need to use the same settings to load the correct object. This can be done:

```
SaveToolboxSystem.Instance.LoadData<List<Vector3>>(fileSaveSettings);
```

The above function call would try to load the custom save data of type List<Vector3>. This should be replaced with the type you are trying to load. Both the TryCreateSaveData and LoadData have async versions.

Scene Saving

There is a setting in the save preferences scriptable object which determines whether or not you would like to save the current scenes in the save data. If selected, when the game is loaded, it will load all previous scenes that were open when it was saved and it will also put all old loadable objects in the correct scenes. It will also find all data to deserialize and put them in the correct objects. This should also work with multiple scene levels that use additive scenes. It is recommended that you handle your own scene loading for most cases as this will just load the scenes and not provide any other custom functionality post.

Extending Serializers

Extending Json Serializer

The Json Serializer has 4 functions you can override to edit the functionality of how it serializes:

- **Bool CanSerialize(Type type)**: This function is what determines whether or not the serializer should go ahead and serialize a type like this. It takes a parameter of type and then inside here you can check the type against any you want to add custom serialization for and if it is of that type return true.

```
protected virtual bool CanSerialize(Type type)
{
    return type == typeof(Vector2) ||
           type == typeof(Vector3) ||
           type == typeof(Vector4) ||
           type == typeof(Color) ||
           type == typeof(Quaternion) ||
           type == typeof(Texture2D) ||
           type.IsEnum ||
           type == typeof(StbByteWriter);
}
```

- **JsonBaseNode Serialize(object objectValue)**: This the function in which you can override how the type that can be serialized is serialized. In this scenario we need to return a JsonBaseNode. These are just nodes that can hold object data in different

ways. They each have a key and a value, with the value that is of type object.

```
public virtual JsonBaseNode Serialize(object objectValue)
{
    var objectNode = new JsonObjectNode();
    switch (objectValue)
    {
        case Vector2 vector2:
        {
            objectNode.AddSimpleChild(name: "X:", vector2.x);
            objectNode.AddSimpleChild(name: "Y:", vector2.y);
            return objectNode;
        }
        case Vector3 vector3:
        {
            objectNode.AddSimpleChild(name: "X:", vector3.x);
            objectNode.AddSimpleChild(name: "Y:", vector3.y);
            objectNode.AddSimpleChild(name: "Z:", vector3.z);
            return objectNode;
        }
        case Vector4 vector4:
        {
            objectNode.AddSimpleChild(name: "X:", vector4.x);
            objectNode.AddSimpleChild(name: "Y:", vector4.y);
            objectNode.AddSimpleChild(name: "Z:", vector4.z);
            objectNode.AddSimpleChild(name: "W:", vector4.w);
            return objectNode;
        }
        case Color color:
        {
            objectNode.AddSimpleChild(name: "R:", color.r);
            objectNode.AddSimpleChild(name: "G:", color.g);
            objectNode.AddSimpleChild(name: "B:", color.b);
            objectNode.AddSimpleChild(name: "A:", color.a);
            return objectNode;
        }
        case Quaternion quaternion:
        {
            objectNode.AddSimpleChild(name: "X:", quaternion.x);
            objectNode.AddSimpleChild(name: "Y:", quaternion.y);
            objectNode.AddSimpleChild(name: "Z:", quaternion.z);
            objectNode.AddSimpleChild(name: "W:", quaternion.w);
            return objectNode;
        }
        case Texture2D texture2D:
        {
            var textureArrayNode = SerializeCollection(typeof(byte[]), objectValue: texture2D.EncodeToPNG());
            textureArrayNode.Name = "Bytes:";
            objectNode.AddChild(textureArrayNode);
            return objectNode;
        }
        case Enum targetEnum:
        {
            objectNode.AddSimpleChild(name: "Enum:", value: Convert.ToInt32(targetEnum));
            return objectNode;
        }
        case StbByteWriter stbByteWriter:
        {
            var arrayNode = SerializeCollection(typeof(byte[]), objectValue: stbByteWriter.GetByteArray());
            arrayNode.Name = "Bytes:";
            objectNode.AddChild(arrayNode);
            return objectNode;
        }
    }
}
```

- **object Deserialize(Type type, compoundNode):** This is how you will deserialize an object. You will receive a compound node and you can get the other nodes off of it by getting the children with the correct label. Below is an example of how it is done. Make su


```

public virtual object Deserialize(Type type, JsonCompoundNode jsonCompoundNode)
{
    if (type == typeof(Vector2))
    {
        var x = (JsonSimpleNode)jsonCompoundNode.GetChild("X:");
        var y = (JsonSimpleNode)jsonCompoundNode.GetChild("Y:");
        if (x.Value is int xInt && y.Value is int yInt)
        {
            return new Vector2(xInt, yInt);
        }

        return new Vector2(x.AsFloat(), y.AsFloat());
    }

    if (type == typeof(Vector3))
    {
        var x = (JsonSimpleNode)jsonCompoundNode.GetChild("X:");
        var y = (JsonSimpleNode)jsonCompoundNode.GetChild("Y:");
        var z = (JsonSimpleNode)jsonCompoundNode.GetChild("Z:");
        if (x.Value is int xInt && y.Value is int yInt && z.Value is int zInt)
        {
            return new Vector3(xInt, yInt, zInt);
        }

        return new Vector3(x.AsFloat(), y.AsFloat(), z.AsFloat());
    }

    if (type == typeof(Vector4))
    {
        var x = (JsonSimpleNode)jsonCompoundNode.GetChild("X:");
        var y = (JsonSimpleNode)jsonCompoundNode.GetChild("Y:");
        var z = (JsonSimpleNode)jsonCompoundNode.GetChild("Z:");
        var w = (JsonSimpleNode)jsonCompoundNode.GetChild("W:");

        if (x.Value is int xInt && y.Value is int yInt && z.Value is int zInt && w.Value is int wInt)
        {
            return new Vector4(xInt, yInt, zInt, wInt);
        }

        return new Vector4(x.AsFloat(), y.AsFloat(), z.AsFloat(), w.AsFloat());
    }
}

```

re to return an object of the intended type.

- **GetFieldsAndProperties(Type objectType):** This is the function you want to override if you want to change how the serializer targets certain fields and properties.

```

public virtual List<MemberInfo> GetFieldsAndProperties(Type objectType)
{
    return StbUtilities.GetAllSerializableFieldsAndProperties(objectType, serializationSettings);
}

```

Extending Binary Serializer

The binary serializer can be extended just like the JSON serializer as described above. As they are both custom serializers created specifically for this tool they share a number of similarities but will still need their own custom extension if required. The one major difference between how this serializes data is it uses the `StbByteWriter` and the `StbByteReader` to store the data in the binary format.

- **Bool CanSerialize(Type type):** This function is what determines whether or not the serializer should go ahead and serialize a type like this. It takes a parameter of type and then inside here you can check the type against any you want to add custom serialization for and if it is of that type return true.

```
protected virtual bool CanSerialize(Type type)
{
    return type == typeof(Vector2) ||
           type == typeof(Vector3) ||
           type == typeof(Vector4) ||
           type == typeof(Color) ||
           type == typeof(Quaternion) ||
           type == typeof(Texture2D) ||
           type.IsEnum ||
           type == typeof(StbByteWriter);
}
```

- **Void Serialize(object objectValue, StbByteWriter byteWriter):** This the function in which you can override how the type that can be serialized is serialized. In this scenario we need to write the object data to the `StbByteWriter`. All primitives and some unity types already have functionality for `Write()` such as vectors, quaternions and color etc. So for example you can just do `stbByteWriter.Write(Vector3)`.

```

protected virtual void Serialize(object objectValue, StbByteWriter byteWriter)
{
    switch (objectValue)
    {
        case Vector2 vector2:
            byteWriter.Write(vector2);
            return;
        case Vector3 vector3:
            byteWriter.Write(vector3);
            return;
        case Vector4 vector4:
            byteWriter.Write(vector4);
            return;
        case Color color:
            byteWriter.Write(color);
            return;
        case Quaternion quaternion:
            byteWriter.Write(quaternion);
            return;
        case Texture2D texture2D:
            var byteArray:byte[] = texture2D.EncodeToPNG();
            byteWriter.Write(byteArray);
            return;
        case Enum targetEnum:
            byteWriter.Write(targetEnum.GetType().AssemblyQualifiedName);
            byteWriter.Write(Convert.ToInt32(targetEnum));
            return;
        case StbByteWriter stbByteWriter:
            byteWriter.Write(stbByteWriter.GetByteArray());
            return;
    }

    throw new Exception(message: $"Could not serialize object of type: {objectValue.GetType()}");
}

```

- **object Deserialize(Type type, compoundNode):** The deserialization function will pass a StbByteReader through the parameters with all the byte data that was passed in when serialized. You will want to read all the data that you wrote and use it to create the object that you serialized. **Please Note:** Because how the reader and writer works you have to read the data in the same order it was written. As whenever any data is read it moves the current internal position of the byte reader.

```
protected virtual object Deserialize(Type type, StbByteReader byteReader)
{
    if (type == typeof(Vector2))
    {
        return byteReader.ReadVector2();
    }

    if (type == typeof(Vector3))
    {
        return byteReader.ReadVector3();
    }

    if (type == typeof(Vector4))
    {
        return byteReader.ReadVector4();
    }

    if (type == typeof(Color))
    {
        return byteReader.ReadColor();
    }

    if (type == typeof(Quaternion))
    {
        return byteReader.ReadQuaternion();
    }

    if (type == typeof(Texture2D))
    {
        var newTexture = new Texture2D(width: 1, height: 1);
        var byteArray:byte[] = byteReader.ReadByteArray();
        newTexture.LoadImage(byteArray);
        return newTexture;
    }

    if (type.IsEnum)
    {
        var enumType = StbSerializationUtilities.GetType(byteReader.ReadString());
        if (enumType == null) throw new Exception(message: "Could not deserialize enum. Have assemblies changed?");

        return Enum.ToObject(enumType, value:byteReader.ReadInt());
    }

    if (type == typeof(StbByteWriter))
    {
        var byteArray:byte[] = byteReader.ReadByteArray();
        return new StbByteReader(byteArray);
    }
}
```

- **GetFieldsAndProperties(Type objectType):** This is the function you want to override if you want to change how the serializer targets certain fields and properties.

```
public virtual List<MemberInfo> GetFieldsAndProperties(Type objectType)
{
    return StbUtilities.GetAllSerializableFieldsAndProperties(objectType, serializationSettings);
}
```

Frequently Asked Questions

- **Can a component that doesn't implement ISaveDataEntity reference be saved?**
No, not currently as it needs a Save Identifier to be able to find it once all data is loaded. Most basic scripts set up in the tool already implement this interface so will work out of the box. Any custom component will just need to implement the ISaveDataEntity interface.
- **Where is my save file stored by default?**
By default, the data path root is Application.PersistentDataPath. This can change on different platforms but on a windows PC it should be in the AppData/LocalLow folder by default. You can also pass in custom paths when saving data.
- **Are multiple saves supported?**
Yes, it is built on a save slot system that allows you to choose which slot the data will be saved to or no slot if you'd prefer (useful for data you want to persist between play sessions such as settings).
- **When I first save or load at runtime there is a lag spike, why is this?**
This is because SaveToolbox will check the assemblies for any custom scripts you have created to implement custom functionality and this is done when you first try to save or load using SaveToolbox at runtime. It is an easy problem to solve however, you just need to call .Initialize() on the SaveToolboxSystem singleton e.g.
SaveToolboxSystem.Instance.Initialize();
Once this function has been called there should no longer be an unexplained lag spike during that play session.
- **Does it support addressables?**
Yes. Just ensure you have it in the project and you can select to use addressables in the SaveToolboxPreferences scriptable object.
- **Which collections are supported currently?**
Array, List, Queue, Stack and Dictionary.
- **Are tuples supported?**
Not yet but I would like to add this functionality in the future.